Labor Verteilte Systeme

Aufgabe 3: Kafka und Eventgetriebene Kommunikation

Implementierung eines Linetris-Clients

Protokoll zur praktischen Umsetzung

Duale Hochschule Baden-Württemberg Mosbach

Studiengang Informatik

Kurs: TINF23B

Dozent: Prof. Dr. rer. nat. Alexander Auch

Abstract

Im Rahmen dieser Arbeit wird ein Kafka-basierter Client für das verteilte Spiel „Linetris“ implementiert. Der Client folgt einer ereignisgesteuerten Architektur, bei der sämtliche Spiellogik durch einen zentralen Game-Master gesteuert wird. Der entwickelte Client übernimmt sowohl die Rolle eines Producers zur Übermittlung von Spielzügen als auch die eines Consumers zur Verarbeitung bestätigter Ereignisse. Der lokale Spielzustand wird ausschließlich anhand vom Game-Master bestätigter Events aufgebaut und aktualisiert. Ziel ist die konsistente, nebenläufige Verarbeitung von Spielzuständen sowie die korrekte Umsetzung der durch Kafka vermittelten Kommunikationssemantik.

Inhaltsverzeichnis

1 Einleitung **1**
1.1 Ziel der Aufgabe . . . 1
1.2 Motivation . . . 1
1.3 Aufbau des Dokuments . . . 2

2 Architektur und Entwurfsentscheidungen **3**
2.1 Event-Driven Architektur . . . 3
2.2 Game-Master als Single Source of Truth . . . 3
2.3 Producer- und Consumer-Trennung . . . 4
2.4 Kafka-Konfiguration . . . 4
2.4.1 Verwendung der `gameId` als Key . . . 4
2.4.2 Consumer Groups pro Client . . . 5
2.4.3 Offset-Handling und Deduplication . . . 5

3 Systemarchitektur und Implementierung **6**
3.1 Paketstruktur . . . 6
3.2 Zentrale Komponenten . . . 7

4 Kafka-Kommunikation und Datenmodell **8**
4.1 Request-Formate . . . 8
4.2 Event-Formate . . . 8
4.3 Verarbeitungssemantik . . . 10

5 Event-Verarbeitung **11**
5.1 Verarbeitungslogik . . . 11
5.2 Nebenläufigkeit . . . 11

6 Test und Nachweise **13**
6.1 Testfälle . . . 13
6.2 Konsolenausgaben . . . 18

7 Erfolgreicher Spielverlauf **20**
7.1 Verwendete gameId . . . 20
7.2 Chronologische Eventliste (realer Verlauf) . . . 20
7.3 Finaler Zustand . . . 20

8 Antworten zu Kafka-Grundlagen **22**
8.1 Datenorganisation in Topics . . . 22
8.2 Partitionierung . . . 22
8.3 Consumer Groups . . . 23
8.4 Vermeidung mehrfacher Verarbeitung . . . 23
8.5 Standalone Consumer . . . 24

9 Probleme und Lösungen **25**

10 Fazit **27**

1 Einleitung

1.1 Ziel der Aufgabe

Ziel dieser Aufgabe ist die Entwicklung eines Clients für das Spiel „Linetris“, welcher in eine Kafka-basierte, ereignisgesteuerte Systemarchitektur integriert ist. Im Gegensatz zu einer klassischen Implementierung wird die vollständige Spiellogik nicht lokal im Client umgesetzt, sondern durch einen externen Game-Master kontrolliert, der als autoritative Instanz fungiert.

Der entwickelte Client übernimmt dabei zwei zentrale Rollen: Einerseits agiert er als Producer, indem Spielzüge in das Kafka-Topic `game-requests` gesendet werden. Andererseits fungiert er als Consumer, der die vom Game-Master erzeugten Ereignisse aus dem Topic `game-events` verarbeitet. Der lokale Spielzustand darf dabei ausschließlich auf Basis bestätigter Ereignisse mit dem Zustand `state = OK` aktualisiert werden, während Ereignisse mit `state = ERROR` lediglich angezeigt, jedoch nicht angewendet werden.

Ein wesentliches funktionales Element ist die Verwendung einer eindeutigen `gameId` in Form einer UUID zur Identifikation einzelner Spiele. Sämtliche Requests und Events müssen eindeutig dieser ID zugeordnet werden, um eine konsistente Zustandsprojektion zu gewährleisten. Darüber hinaus muss der Client in der Lage sein, parallel in mehreren Instanzen ausgeführt zu werden, um die Rollen von `PLAYER1` und `PLAYER2` abzubilden.

Die Aufgabe umfasst somit nicht die Implementierung der Spiellogik selbst, sondern die korrekte Integration in ein verteiltes, eventgetriebenes System unter Verwendung von Apache Kafka, einschließlich Zustandsmanagement, Nebenläufigkeit und robuster Ereignisverarbeitung.

1.2 Motivation

Die vorliegende Aufgabe dient der praktischen Auseinandersetzung mit zentralen Konzepten verteilter Systeme und insbesondere der Event-Driven Architecture. Moderne Softwaresysteme sind zunehmend durch lose Kopplung, asynchrone Kommunikation und skalierbare Datenströme geprägt. Apache Kafka stellt hierbei eine zentrale Infrastrukturkomponente dar, die als verteilter Commit-Log die Grundlage für robuste, fehlertolerante und hochskalierbare Systeme bildet.

Im Kontext dieser Aufgabe wird dieses Paradigma konkret angewendet: Der Game-Master fungiert als Single Source of Truth, während der Client lediglich eine lokale Projektion des Systemzustands auf Basis eingehender Ereignisse aufbaut. Dieses Muster entspricht etablierten architektonischen Ansätzen wie Event Sourcing und CQRS, bei denen der Zustand nicht direkt manipuliert, sondern aus einer Folge von Ereignissen rekonstruiert wird.

Besonders relevant ist hierbei die klare Trennung zwischen Befehlen (Commands) und Ereignissen (Events). Während der Client lediglich Intentionen in Form von Requests äußert, entscheidet ausschließlich der Game-Master über deren Gültigkeit und erzeugt entsprechende Events. Dadurch wird sichergestellt, dass Inkonsistenzen, Race Conditions oder fehlerhafte Zustandsübergänge vermieden werden.

Darüber hinaus verdeutlicht die Aufgabe zentrale Herausforderungen verteilter Systeme, wie beispielsweise:

- Asynchrone Kommunikation und zeitverzögerte Rückmeldungen
- Konsistenz und Zustandsrekonstruktion über Event-Streams
- Partitionierung und Reihenfolgegarantien in Kafka
- Nebenläufigkeit durch parallele Verarbeitung von Eingaben und Events

Die Umsetzung eines funktionalen Clients in diesem Kontext stellt somit eine realitätsnahe Übung dar, die grundlegende Prinzipien moderner, skalierbarer Systemarchitekturen vermittelt.

1.3 Aufbau des Dokuments

Das vorliegende Protokoll ist systematisch aufgebaut, um sowohl die fachlichen Anforderungen als auch deren konkrete Umsetzung nachvollziehbar darzustellen.

Zunächst werden die Anforderungen der Aufgabenstellung analysiert und strukturiert aufbereitet. Darauf aufbauend werden die wesentlichen Architekturentscheidungen erläutert und begründet, insbesondere im Hinblick auf die ereignisgetriebene Kommunikation und die Rolle des Game-Masters als zentrale Instanz.

Im Anschluss wird das zugrunde liegende Kafka-Kommunikationsmodell beschrieben, gefolgt von einer detaillierten Darstellung des Datenmodells sowie der Verarbeitung eingehender Ereignisse. Die konkrete Implementierung wird anhand der Paket- und Klassenstruktur systematisch eingeordnet.

Ein weiterer Schwerpunkt liegt auf der Validierung der Lösung. Hierzu werden Testfälle definiert und durch geeignete Konsolenausschnitte und Event-Sequenzen nachgewiesen. Ergänzend werden die in der Aufgabenstellung geforderten theoretischen Fragen zu Kafka und Partitionierung beantwortet.

Abschließend werden aufgetretene Probleme sowie deren Lösungen reflektiert und die Ergebnisse in einem Fazit zusammengefasst.

2 Architektur und Entwurfsentscheidungen

2.1 Event-Driven Architektur

Die Implementierung folgt bewusst einer ereignisgetriebenen Architektur, da die Aufgabenstellung selbst eine asynchrone Kommunikation zwischen Client, Kafka und Game-Master vorgibt. Der Client sendet keine direkten Methodenaufrufe an eine lokale Spiellogik, sondern veröffentlicht Spielabsichten als Nachrichten im Topic `game-requests`. Die eigentliche Reaktion des Systems erfolgt erst später über ein vom Game-Master veröffentlichtes Ereignis im Topic `game-events`. Damit ist der Ablauf nicht synchron im Sinne eines unmittelbaren Funktionsrückgabewertes, sondern ereignisbasiert und zeitlich entkoppelt.

Diese Entkopplung ist fachlich wichtig, weil ein Spielzug zunächst nur eine Absicht darstellt. Ein Benutzer kann zwar eingeben, dass ein Stein in eine bestimmte Spalte gelegt werden soll, daraus folgt aber noch nicht, dass dieser Zug gültig ist. Erst der Game-Master prüft, ob der Zug regelkonform ist, und publiziert anschließend entweder ein bestätigtes Ereignis mit `state = OK` oder eine Fehlermeldung mit `state = ERROR`. Der Client reagiert daher nicht auf seine eigene Eingabe als Wahrheit, sondern ausschließlich auf bestätigte Ereignisse.

Die Umsetzung dieses Prinzips zeigt sich insbesondere in der Trennung zwischen `SubmitMoveUseCase` und `ApplyGameEventUseCase`. Der `SubmitMoveUseCase` sendet lediglich einen Request an Kafka:

```
public void execute(PlaceMoveCommand command) {
    var dto = mapper.toPlaceMoveRequest(command);
    producer.send(dto, command.getGameId().toString());
}
```

Listing 1: Senden eines Spielzugs als Command

Der lokale Zustand wird an dieser Stelle bewusst nicht verändert. Die Zustandsänderung erfolgt erst im `ApplyGameEventUseCase`, nachdem ein bestätigtes Event konsumiert und auf ein Domain Event abgebildet wurde. Dadurch wird verhindert, dass der Client einen Zug lokal anwendet, der vom Game-Master später abgelehnt wird.

2.2 Game-Master als Single Source of Truth

Der Game-Master wird in der Architektur als Single Source of Truth behandelt. Das bedeutet, dass der Game-Master die einzige autoritative Instanz für die Gültigkeit von Spielzügen, die Gewinnentscheidung und Spezialaktionen wie `deleteBottomRow` ist. Der lokale Client hält lediglich eine abgeleitete Sicht auf den Spielzustand. Diese Sicht ist keine eigene Wahrheit, sondern eine Projektion der bestätigten Ereignisse aus `game-events`.

Diese Entscheidung folgt direkt aus der Aufgabenstellung: Die Spiellogik soll nicht im Client implementiert werden. Würde der Client beispielsweise selbst prüfen, ob ein Spieler gewonnen hat, könnten mehrere Clients zu unterschiedlichen Ergebnissen kommen. Dies wäre insbesondere bei asynchroner Kommunikation problematisch, da Clients Nachrichten zeitversetzt empfangen können. Ein Client könnte auf Basis eines noch unvollständigen lokalen Zustands einen Gewinn erkennen, obwohl der Game-Master den kausal korrekten Spielverlauf anders bewertet.

Daher enthält der Client keine lokale Gewinnlogik. Die Klasse `GameStateProjector` setzt den Status des Spiels erst dann auf `WON`, wenn ein entsprechendes `GameWonEvent` aus dem Event-Stream erzeugt wurde:

```
} else if (event instanceof GameWonEvent) {
    GameWonEvent e = (GameWonEvent) event;

    state.setStatus(GameStatus.WON);
    state.setWinner(e.getPlayer());
}
```

Listing 2: Gewinnzustand wird nur durch ein bestätigtes Event gesetzt

Der Client berechnet also nicht, ob vier Steine in einer Reihe liegen. Er übernimmt lediglich die vom Game-Master bestätigte Entscheidung. Dadurch wird vermieden, dass mehrere Instanzen unabhängig voneinander widersprüchliche Spielzustände erzeugen.

2.3 Producer- und Consumer-Trennung

Die Implementierung trennt Producer- und Consumer-Verantwortlichkeiten klar voneinander. Der Producer ist ausschließlich dafür zuständig, Requests an das Topic `game-requests` zu senden. Der Consumer liest ausschließlich Events aus `game-events`. Diese Trennung entspricht der fachlichen Differenz zwischen Commands und Events: Commands drücken eine Absicht aus, Events beschreiben bestätigte Fakten.

In der Implementierung ist diese Trennung strukturell sichtbar. Der `KafkaGameCommandProducer` kapselt das Senden von Nachrichten, während der `KafkaGameEventConsumer` das Lesen aus Kafka übernimmt. Die fachliche Verarbeitung der eingehenden Events liegt wiederum im `KafkaConsumerLoop`, im `EventMapper` und im `ApplyGameEventUseCase`. Dadurch wird verhindert, dass Sende- und Empfangslogik vermischt werden.

Diese Struktur ist bewusst nicht übermäßig komplex. Es wurde kein zusätzliches Framework für Event Processing, keine lokale Datenbank und kein eigener Microservice eingeführt. Für die Aufgabenstellung reicht ein modularer Java-Client aus, der intern sauber getrennte Komponenten besitzt. Die Architektur bleibt dadurch nachvollziehbar, prüfbar und wartbar, ohne unnötige Infrastruktur aufzubauen.

2.4 Kafka-Konfiguration

2.4.1 Verwendung der `gameId` als Key

Für alle gesendeten Requests wird die `gameId` als Kafka-Key verwendet. Diese Entscheidung ist zentral, weil Kafka Reihenfolgegarantien nur innerhalb einer Partition gibt. Werden alle Nachrichten eines Spiels mit derselben `gameId` als Key gesendet, landen sie deterministisch in derselben Partition. Dadurch bleibt die Reihenfolge der Spielzüge für ein konkretes Spiel erhalten.

Dies ist in den UseCases explizit umgesetzt:

```
producer.send(dto, command.getGameId().toString());
```

Listing 3: gameId als Kafka-Key

Ohne Key könnten Nachrichten über verschiedene Partitionen verteilt werden. Dann wäre nicht garantiert, dass ein Client die Events eines Spiels in der kausal richtigen Reihenfolge verarbeitet. Gerade bei `deleteBottomRow` wäre das kritisch, weil diese Aktion vom vorherigen Board-Zustand abhängt.

2.4.2 Consumer Groups pro Client

Jede Client-Instanz verwendet eine eigene Consumer Group:

```
props.put(ConsumerConfig.GROUP_ID_CONFIG, "game-client-" + UUID.
    randomUUID());
```

Listing 4: Eindeutige Consumer Group pro Client-Start

Diese Entscheidung ist notwendig, weil beide Spieler-Clients alle relevanten Events sehen müssen. Würden beide Clients dieselbe Consumer Group verwenden, würde Kafka die Partitionen zwischen ihnen aufteilen. Dann würde jeder Client nur einen Teil des Event-Streams erhalten. Für einen Spielclient wäre das falsch, weil jeder Client sein lokales Spielbrett vollständig rekonstruieren muss.

Durch eine eindeutige Group-ID pro Client-Start erhält jede Client-Instanz ihren eigenen vollständigen Konsum des Topics. Die Filterung nach `gameId` erfolgt anschließend lokal im `KafkaConsumerLoop`. Damit wird erreicht, dass mehrere Client-Instanzen parallel betrieben werden können, ohne sich gegenseitig Events wegzunehmen.

2.4.3 Offset-Handling und Deduplication

Das Offset-Handling erfolgt manuell, indem `ENABLE_AUTO_COMMIT_CONFIG` deaktiviert ist und nach der Verarbeitung explizit committet wird:

```
props.put(ConsumerConfig.ENABLE_AUTO_COMMIT_CONFIG, false);
...
consumer.commit();
```

Listing 5: Manuelles Offset-Commit

Zusätzlich enthält der Consumer eine defensive Deduplizierung über das Tripel aus Topic, Partition und Offset:

```
String recordId = record.topic() + "-" + record.partition() + "-" +
    record.offset();

if (!processedRecords.add(recordId)) {
    continue;
}
```

Listing 6: Defensive Deduplizierung über topic-partition-offset

Diese Maßnahme ist defensiv. Kafka liefert einem korrekt laufenden Consumer eine Nachricht innerhalb einer Partition sequenziell über ihren Offset. Dennoch ist eine solche Absicherung sinnvoll, weil bei Fehlern, Rebalancing oder Neustarts grundsätzlich Mehrfachverarbeitung möglich sein kann. Würde ein `MovePlacedEvent` doppelt angewendet, würde der Client zweimal einen Stein in dieselbe Spalte setzen und dadurch ein falsches Board projizieren. Die Deduplizierung verhindert genau diesen Fehlerfall innerhalb einer laufenden Client-Instanz.

3 Systemarchitektur und Implementierung

3.1 Paketstruktur

Die Implementierung ist in mehrere fachlich getrennte Pakete gegliedert. Ziel dieser Struktur ist eine klare Trennung zwischen Benutzereingabe, Anwendungsfällen, Domänenmodell, Kafka-Infrastruktur und Nachrichtentransformation.

Paket	**Verantwortung**
`aufgabe3.bootstrap`	Composition Root der Anwendung: Instanziierung und Verdrahtung zentraler Komponenten sowie Bereitstellung des Einstiegspunkts.
`aufgabe3.client`	Steuerung des Clients, Konsoleneingabe, lokaler Spielkontext und Thread-Start.
`aufgabe3.application.usecase`	Anwendungsfälle zum Starten eines Spiels, Senden eines Spielzugs und Anwenden eingehender Events.
`aufgabe3.domain.command`	Fachliche Commands wie `StartGameCommand` und `PlaceMoveCommand`.
`aufgabe3.domain.event`	Fachliche Events wie `GameStartedEvent`, `MovePlacedEvent`, `BottomRowDeletedEvent` und `GameWonEvent`.
`aufgabe3.domain.model`	Lokales Spielmodell mit `GameState`, `Board`, `Player` und `GameStatus`.
`aufgabe3.domain.service`	Projektion bestätigter Events auf den lokalen Zustand durch den `GameStateProjector`.
`aufgabe3.infrastructure.kafka`	Kafka-spezifische Producer-, Consumer- und Konfigurationslogik.
`aufgabe3.infrastructure.repository`	In-Memory-Speicherung des lokal projizierten Spielzustands.
`aufgabe3.messaging.dto`	DTOs für die JSON-Kommunikation mit Kafka.
`aufgabe3.messaging.mapper`	Transformation zwischen DTOs und fachlichen Commands beziehungsweise Events.

Diese Paketstruktur vermeidet eine monolithische Vermischung aller Aufgaben in einer einzigen Klasse. Gleichzeitig bleibt sie bewusst schlank. Eine zusätzliche Datenbankschicht oder ein komplexes Framework wäre für diese Aufgabe nicht angemessen, weil Kafka bereits als zentrales Event-Log dient und der Client nur eine temporäre lokale Projektion benötigt.

3.2 Zentrale Komponenten

Die zentrale Einstiegskomponente ist `GameClient`. Diese Klasse verdrahtet Producer, Consumer, Mapper, UseCases, Repository und Threads. Sie übernimmt damit die Rolle eines einfachen Composition Roots. Die fachliche Logik liegt nicht in `GameClient`, sondern in spezialisierten Komponenten.

```java
public void start() {
    consumerThread.start();

    inputThread.start();

    consumerLoop.awaitReady();
}
```

Listing 7: Start von Consumer- und Input-Thread

Der `ConsoleInputController` verarbeitet die Konsoleneingaben. Er entscheidet, ob eine neue `gameId` erzeugt oder eine bestehende `gameId` verwendet wird. Anschließend sendet er Spielzüge über den `SubmitMoveUseCase`. Wichtig ist, dass der Controller nach Eingabe eines Spielzugs nicht selbst das Board verändert.

Der `KafkaConsumerLoop` ist für die kontinuierliche Verarbeitung eingehender Events zuständig. Er liest Events aus Kafka, filtert nach der aktuellen `gameId`, behandelt Fehlerereignisse und übergibt bestätigte Events an den `ApplyGameEventUseCase`. Damit bildet er die Brücke zwischen Kafka-Infrastruktur und fachlicher Projektion.

Der `GameStateProjector` aktualisiert den lokalen Zustand deterministisch. Ein `MovePlacedEvent` wird angewendet, indem der Stein in der niedrigsten freien Zelle der jeweiligen Spalte platziert wird. Ein `BottomRowDeletedEvent` verschiebt alle darüberliegenden Zeilen nach unten. Ein `GameWonEvent` setzt den Status auf `WON`. Die Projektionslogik ist damit bewusst einfach und ausschließlich eventbasiert.

4 Kafka-Kommunikation und Datenmodell

4.1 Request-Formate

Der Client sendet zwei Arten von Requests an das Topic `game-requests`. Dabei handelt es sich ausschließlich um fachliche Commands, also um die Beschreibung einer beabsichtigten Aktion, jedoch noch nicht um einen tatsächlich bestätigten Zustand.

Der erste Request startet ein neues Spiel. Hierbei wird eine neue UUID als `gameId` erzeugt und zusammen mit den beteiligten Clients und Spielern übertragen.

```
{
  "type": "newGame",
  "gameId": "a17c0b2f-1263-445d-b2bc-87ad89e2b3a6",
  "client1": { "name": "Client" },
  "player1": { "name": "Player1" },
  "client2": { "name": "Client" },
  "player2": { "name": "Player2" }
}
```

Listing 8: Beispiel für einen newGame-Request

Der zweite Request beschreibt einen Spielzug. Er enthält die `gameId`, den Spieler sowie die gewünschte Spalte.

```
{
  "type": "move",
  "gameId": "a17c0b2f-1263-445d-b2bc-87ad89e2b3a6",
  "player": "PLAYER1",
  "column": 1
}
```

Listing 9: Beispiel für einen move-Request

Fachlich stellen diese Requests keine Zustandsänderungen dar, sondern lediglich Absichten. Ob und in welcher Form daraus tatsächliche Zustandsänderungen entstehen, wird ausschließlich durch den Game-Master entschieden.

Zur Sicherstellung einer korrekten Event-Reihenfolge wird die `gameId` als Kafka-Key verwendet. Dadurch werden alle Nachrichten eines Spiels derselben Partition zugeordnet, wodurch die Reihenfolge der Events innerhalb eines Spiels garantiert erhalten bleibt.

4.2 Event-Formate

Die Verarbeitung der tatsächlichen Zustandsänderungen erfolgt über das Topic `game-events`. Jede empfangene Nachricht enthält ein Event-Envelop mit den zentralen Feldern `gameId`, `state` und `actions`.

Das Feld `state` fungiert dabei als Gatekeeper und muss stets zuerst ausgewertet werden. Nur wenn `state = OK` ist, dürfen die enthaltenen Aktionen auf das lokale Modell angewendet werden. Bei `state = ERROR` wird das Ereignis lediglich angezeigt, jedoch nicht verarbeitet.

```
{
  "timeStamp": 1746130002467,
  "actions": [
    {
```

```
      "type": "move",
      "player": "PLAYER1",
      "column": 1
    }
  ],
  "state": "OK",
  "gameId": "a17c0b2f-1263-445d-b2bc-87ad89e2b3a6"
}
```

Listing 10: Beispiel für ein bestätigtes OK-Event

```
{
  "timeStamp": 1746130691239,
  "actions": [
    {
      "type": "move",
      "player": "PLAYER1",
      "column": 11
    }
  ],
  "state": "ERROR",
  "message": "Illegal column number: 11, max=8",
  "gameId": "a17c0b2f-1263-445d-b2bc-87ad89e2b3a6"
}
```

Listing 11: Beispiel für ein ERROR-Event

Ein wesentliches Merkmal des Datenmodells ist, dass ein einzelnes Kafka-Event mehrere Aktionen enthalten kann. Diese Aktionen werden im Client in einzelne fachliche Events transformiert und anschließend sequenziell verarbeitet. Ein Kafka-Event entspricht somit nicht direkt einer einzelnen Zustandsänderung, sondern einer Folge deterministischer Zustandsübergänge.

Der Client führt dabei keinerlei eigene Spiellogik aus. Stattdessen werden ausschließlich die vom Game-Master gelieferten Aktionen in der gegebenen Reihenfolge auf das lokale Modell projiziert.

Ein Beispiel für mehrere Aktionen innerhalb eines Events ist die Kombination eines Spielzugs mit dem Löschen der untersten Reihe:

```
{
  "timeStamp": 1746130485596,
  "actions": [
    {
      "type": "move",
      "player": "PLAYER1",
      "column": 8
    },
    {
      "type": "deleteBottomRow",
      "row": 8
    }
  ],
  "state": "OK",
  "gameId": "a17c0b2f-1263-445d-b2bc-87ad89e2b3a6"
}
```

Listing 12: Beispiel für move mit deleteBottomRow

Das `deleteBottomRow`-Ereignis stellt dabei eine nicht lokal ableitbare Zustandsänderung dar. Der Client besitzt keine eigene Spiellogik zur Berechnung solcher Effekte und ist daher vollständig auf die korrekte Sequenz der vom Game-Master gelieferten Aktionen angewiesen.

Ein Gewinn wird durch eine `winAction` signalisiert. Diese führt zu einer expliziten Zustandsänderung im lokalen Modell, indem der Spielstatus auf `WON` gesetzt und der Gewinner gespeichert wird.

```
{
  "timeStamp": 1746130584078,
  "actions": [
    {
      "type": "move",
      "player": "PLAYER2",
      "column": 1
    },
    {
      "type": "winAction",
      "player": "PLAYER2"
    }
  ],
  "state": "OK",
  "gameId": "a17c0b2f-1263-445d-b2bc-87ad89e2b3a6"
}
```

Listing 13: Beispiel für ein Gewinn-Event

4.3 Verarbeitungssemantik

Aufgrund der Verarbeitungssemantik von Kafka können Events mehrfach empfangen werden (*at-least-once delivery*). Der Client implementiert daher eine deduplizierende Verarbeitung auf Basis von Topic, Partition und Offset, um sicherzustellen, dass jede Nachricht nur einmal auf das lokale Modell angewendet wird.

Die gesamte Zustandsrekonstruktion erfolgt somit deterministisch durch sequenzielles Anwenden der vom Game-Master gelieferten Events. Der Client fungiert ausschließlich als projektionales Read-Model ohne eigene Entscheidungslogik.

5 Event-Verarbeitung

5.1 Verarbeitungslogik

Die Verarbeitung eingehender Kafka-Events erfolgt in mehreren klar getrennten Schritten. Zunächst werden Nachrichten aus dem Topic `game-events` gelesen. Anschließend wird geprüft, ob das Event zur aktuell verwendeten `gameId` gehört. Events anderer Spiele werden ignoriert, da das Topic mehrere Spiele enthalten kann.

```
if (currentGameId == null || !currentGameId.equals(dto.getGameId())) {
    continue;
}
```

Listing 14: Filterung auf die aktuelle gameId

Danach wird der Zustand des Events ausgewertet. Fehlerereignisse werden immer angezeigt, aber niemals auf das lokale Modell angewendet:

```
if ("ERROR".equals(dto.getState())) {
    eventMapper.printError(dto);
    continue;
}
```

Listing 15: ERROR-Events anzeigen, aber nicht anwenden

Diese Reihenfolge ist entscheidend. Würde der Client zuerst die `actions` auswerten und erst danach den Zustand prüfen, könnte ein abgelehnter Spielzug versehentlich lokal angewendet werden. Genau das verhindert die aktuelle Implementierung.

Nur wenn das Event den Zustand `OK` besitzt, wird es über den `EventMapper` in fachliche Domain Events übersetzt:

```
List<GameEvent> events = eventMapper.map(dto);

for (GameEvent event : events) {
    useCase.execute(event);
}
```

Listing 16: Mapping bestätigter Events auf Domain Events

Die eigentliche Zustandsänderung erfolgt anschließend im `GameStateProjector`. Dadurch bleibt die Verarbeitung deterministisch: Derselbe Event-Stream führt bei gleicher Reihenfolge immer zum gleichen lokalen Spielzustand.

5.2 Nebenläufigkeit

Die Aufgabenstellung empfiehlt eine Lösung mit mehreren Threads, weil Benutzereingaben und Event-Verarbeitung zeitlich unabhängig voneinander stattfinden. Diese Anforderung wird durch zwei getrennte Threads umgesetzt: einen `console-input`-Thread und einen `game-event-consumer`-Thread.

Der Input-Thread wartet blockierend auf Benutzereingaben über die Konsole. Würde derselbe Thread gleichzeitig für Kafka-Polling zuständig sein, könnten während einer blockierenden Eingabe keine Events verarbeitet werden. Eingehende Bestätigungen, Fehlermeldungen oder Gewinnereignisse würden dann erst verzögert oder gar nicht sichtbar. Umgekehrt dürfte das Polling nicht die Eingabe blockieren.

Die Trennung wird in `GameClient` explizit umgesetzt:

```java
this.consumerThread = new Thread(consumerLoop, "game-event-consumer");
this.inputThread = new Thread(inputController, "console-input");
```

Listing 17: Getrennte Threads für Eingabe und Event-Verarbeitung

Da beide Threads auf gemeinsamen Kontext zugreifen, wird der aktuelle Spielkontext über thread-sichere Strukturen verwaltet. In `GameContext` werden `AtomicReference` und `CountDownLatch` verwendet:

```java
private final AtomicReference<UUID> currentGameId = new
    AtomicReference<>();
private final AtomicReference<Player> localPlayer = new
    AtomicReference<>();
private final CountDownLatch gameIdLatch = new CountDownLatch(1);
```

Listing 18: Thread-sicherer Spielkontext

Der `CountDownLatch` stellt sicher, dass der Consumer erst mit der fachlichen Verarbeitung beginnt, wenn eine `gameId` gesetzt wurde. Dadurch wird verhindert, dass Events verarbeitet werden, bevor der Client weiß, welchem Spiel er zugeordnet ist.

Diese Nebenläufigkeit ist für diese Aufgabe ausreichend und bewusst begrenzt. Es wurde kein komplexes Actor-System, kein Scheduler und kein zusätzliches Framework eingesetzt. Die Lösung adressiert genau die notwendige Parallelität zwischen Konsoleneingabe und Kafka-Event-Verarbeitung, ohne unnötige Komplexität einzuführen.

6 Test und Nachweise

6.1 Testfälle

Die Testfälle prüfen gezielt die Teile des Clients, die laut Aufgabenstellung in dessen Verantwortung liegen. Nicht getestet wird die eigentliche Spiellogik von Linetris, also insbesondere nicht die Entscheidung, ob ein Zug gültig ist, ob ein Spieler gewonnen hat oder ob eine Zeile gelöscht werden muss. Diese Entscheidungen liegen beim Game-Master. Der Client wird daher als lokales Read-Model verstanden: Er sendet Spielabsichten als Requests, konsumiert anschließend bestätigte Events und rekonstruiert daraus seinen lokalen Zustand.

Die zentrale Prüffrage lautet somit nicht: „Erkennt der Client korrekte Spielzüge?“, sondern: „Projiziert der Client die vom Game-Master bestätigten Events korrekt, deterministisch und ohne eigene fachliche Entscheidungslogik?“. Diese Abgrenzung ist wesentlich, da die Aufgabenstellung ausdrücklich verlangt, dass Spielzüge erst nach Bestätigung durch den Game-Master als gültig gelten.

- **Spielstart: `GameStartedEvent`**
  Der Spielstart wird durch den Testfall für `GameStartedEvent` geprüft. Der Test erzeugt ein bestätigtes Spielstart-Event und wendet dieses auf ein zunächst leeres Repository an.

  Geprüft wird konkret:

  - Es wird erst nach einem bestätigten Start-Event ein lokaler Spielzustand angelegt.
  - Die vom Event gelieferten Dimensionen werden korrekt übernommen.
  - Das Board ist initial leer.
  - Der Spielstatus ist initial `RUNNING`.

  Warum dieser Test relevant ist:
  Der Client darf keinen Spielzustand auf Basis einer bloßen lokalen Eingabe als endgültig betrachten. Erst die Bestätigung des Game-Masters initialisiert das lokale Modell. Damit wird die Rolle des Game-Masters als autoritative Instanz eingehalten.

  Beispielhafte Konsolenausgabe:

```
=== EVENT APPLIED: GameStartedEvent ===
GameId: c45d77a2-3712-471e-bc33-f3b45aa0fd6e
Status: RUNNING
Board:
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
```

- **Gültige Züge: `MovePlacedEvent`**
  Die Verarbeitung bestätigter Spielzüge wird über `MovePlacedEvent` getestet. Dabei wird geprüft, ob ein vom Game-Master bestätigter Zug korrekt auf das lokale Board projiziert wird.

  Geprüft wird konkret:

  - Ein Stein wird in der angegebenen Spalte platziert.
  - Die Spaltennummer aus dem Event wird korrekt von 1-basiert auf 0-basiert übersetzt.
  - Der Stein fällt an die tiefste freie Position der Spalte.
  - Mehrere Züge in derselben Spalte stapeln sich deterministisch von unten nach oben.

  Warum dieser Test relevant ist:
  Der Client berechnet nicht, ob der Zug erlaubt ist. Er setzt nur die Tatsache um, dass der Game-Master diesen Zug bereits akzeptiert hat. Damit wird die Trennung zwischen Command und Event sichtbar: Der Request ist nur eine Absicht, das Event ist der bestätigte Fakt.

  Beispiel nach zwei bestätigten Zügen in derselben Spalte:

  ```
  Board:
  . . . . . .
  . . . . . .
  . . . . . .
  . . . . . .
  X . . . . .
  X . . . . .
  ```

- **Ungültige Züge: `ErrorHandlingTest`**
  Der Test `ErrorHandlingTest` prüft, dass Events mit `state = ERROR` nicht in fachliche Domain-Events transformiert werden.

  Geprüft wird konkret:

  - Ein ERROR-Event erzeugt keine `MovePlacedEvent`-Instanz.
  - Ein formal gültig aussehender Action-Eintrag wird trotzdem nicht angewendet.
  - Die Entscheidung, ob ein Spielzug ungültig ist, bleibt vollständig beim Game-Master.

  Warum dieser Test relevant ist:
  Die Aufgabenstellung weist ausdrücklich darauf hin, dass zuerst das Feld `state` auszuwerten ist. Ein ERROR-Event darf nicht dadurch gefährlich werden, dass sein `actions`-Feld trotzdem wie ein gültiger Zug aussieht. Genau diese Fehlerklasse wird hier ausgeschlossen.

  Formale Eigenschaft:

```
mapper.map(ERROR_EVENT) -> []
```

Falsches Verhalten wäre:

```
state = ERROR, actions = [{type: "move", column: 1}]
-> MovePlacedEvent wird trotzdem erzeugt
-> lokales Board enthält einen abgelehnten Stein
```

- **Reihenlöschung: `RowDeletionTest`**
  Der Test `RowDeletionTest` prüft die Verarbeitung eines `BottomRowDeletedEvent`. Dazu wird zunächst eine untere Reihe mit Spielsteinen gefüllt und anschließend ein Lösch-Event angewendet.

  Geprüft wird konkret:

  - Die unterste Reihe wird nach dem Event geleert.
  - Darüberliegende Reihen werden nach unten verschoben.
  - Die obere Reihe wird anschließend frei.
  - Die Reihenlöschung wird nicht lokal entschieden, sondern nur als empfangene Game-Master-Aktion umgesetzt.

  Warum dieser Test relevant ist:
  `deleteBottomRow` ist anspruchsvoller als ein einzelner Zug, weil nicht nur ein Feld, sondern die gesamte vertikale Struktur des Boards verändert wird. Der Test zeigt daher, dass der Client auch zusammengesetzte Zustandsänderungen aus dem Event-Stream korrekt projizieren kann.

  Beispielhafte Projektion:

```
Vorher:
. . . .
. . . .
. . . .
X X X X

Nach BottomRowDeletedEvent:
. . . .
. . . .
. . . .
. . . .
```

- **Spielgewinn: `WinConditionTest`**
  Der Test `WinConditionTest` prüft, ob ein `GameWonEvent` korrekt auf den lokalen Spielstatus angewendet wird.

  Geprüft wird konkret:

  - Der Status wird auf `WON` gesetzt.
  - Der Gewinner wird aus dem Event übernommen.

- Der Client erkennt den Gewinn nicht selbst durch Board-Analyse.

Warum dieser Test relevant ist:
Die Gewinnlogik gehört laut Aufgabenstellung zum Game-Master. Der Client darf daher nicht versuchen, vier Steine selbst zu erkennen. Er übernimmt ausschließlich das bestätigte `winAction`-Ereignis.

Beispielhafte Ausgabe:

```
=== EVENT APPLIED: GameWonEvent ===
Status: WON
WINNER: PLAYER1
```

- **Deterministische Rekonstruktion: `DeterminismTest`**
  Der Test `DeterminismTest` spielt denselben geordneten Event-Stream gegen zwei voneinander unabhängige Repositories ab.

  Geprüft wird konkret:

  - Beide Repositories starten leer.
  - Beide erhalten exakt dieselbe Event-Sequenz.
  - Beide erzeugen am Ende exakt denselben Board-Zustand.
  - Die Projektion ist unabhängig von Objektidentitäten oder zufälligem Laufzeitverhalten.

  Warum dieser Test relevant ist:
  Dies ist die zentrale Eigenschaft eines eventgetriebenen Read-Models: Der Zustand ergibt sich deterministisch aus der geordneten Folge bestätigter Events. Damit lässt sich der lokale Zustand aus dem Event-Stream rekonstruieren.

  Formale Charakterisierung:

  ```
  state = fold(orderedEvents)
  ```

  Für zwei unabhängige Repositories gilt:

  ```
  fold(events, repositoryA) == fold(events, repositoryB)
  ```

- **Ordering-Grenzfall: `OrderingTest`**
  Der Test `OrderingTest` wendet absichtlich ein `MovePlacedEvent` vor einem `GameStartedEvent` an.

  Geprüft wird konkret:

  - Ein Zug vor Initialisierung des Spiels erzeugt keinen Zustand.
  - Das Board wird erst nach `GameStartedEvent` angelegt.
  - Der vorherige Zug wird nicht nachträglich angewendet.

Warum dieser Test relevant ist:
Der Test macht sichtbar, dass die Projektion kausal von der Event-Reihenfolge abhängt. In der realen Kafka-Kommunikation wird diese Reihenfolge pro Spiel durch die Verwendung der `gameId` als Kafka-Key unterstützt. Kafka garantiert keine globale Ordnung über das gesamte Topic, aber eine Ordnung innerhalb einer Partition. Deshalb ist die konstante Verwendung derselben `gameId` als Key für alle Events eines Spiels fachlich entscheidend.

Falsche Reihenfolge:

```
MovePlacedEvent(gameId, PLAYER1, column=1)
GameStartedEvent(gameId, rows=6, cols=6)
```

Erwarteter Zustand:

```
Anzahl Steine in column=1: 0
```

- **Bewusste Abgrenzung: `DeduplicationTest`**
  Der Test `DeduplicationTest` dokumentiert bewusst, dass zwei identische `MovePlacedEvent`-Objekte zweimal angewendet werden.

  Geprüft wird konkret:

  - Der Client behandelt jedes empfangene OK-Event als eigenständigen Fakt.
  - Es wird keine nicht spezifizierte Deduplizierungslogik im Domain-Projektor versteckt.
  - Zwei gleiche Events führen zu zwei Zustandsübergängen.

  Warum dieser Test relevant ist:
  Die Aufgabenstellung definiert keine stabile Event-ID, keine Sequenznummer und keine fachliche Regel zur Deduplizierung von Events. Eine echte Idempotenzprüfung wäre daher ohne zusätzliche Event-Identität nicht sauber möglich. Der Test ist deshalb keine Forderung nach Duplicate Handling, sondern eine bewusste Dokumentation der Architekturgrenze: Der Client projiziert bestätigte Events; er entscheidet nicht, ob zwei gleich aussehende Events fachlich dasselbe Ereignis sind.

  Beispiel:

  ```
  MovePlacedEvent(gameId, PLAYER1, column=1)
  MovePlacedEvent(gameId, PLAYER1, column=1)

  Erwartung: 2 Steine in column=1
  ```

  Diese Abgrenzung ist wichtig, weil eine naive Deduplizierung auf Basis des Inhalts falsch wäre. Zwei aufeinanderfolgende gleiche Züge können in einem Event-Stream technisch gleich aussehen, aber fachlich zwei verschiedene bestätigte Ereignisse sein.

- **Längerer Systemtest: `ChaosIntegrationTest`**
  Der `ChaosIntegrationTest` kombiniert mehrere fachliche und technische Eigenschaften in einer längeren Sequenz.

  Geprüft wird konkret:

  - Spielstart und Board-Initialisierung.
  - Mehrere bestätigte Züge beider Spieler.
  - Wiederholte Anwendung gleich aussehender Events als eigenständige Fakten.
  - Reihenlöschung über `BottomRowDeletedEvent`.
  - Weitere Stapelung nach der Reihenlöschung.
  - Spielende durch `GameWonEvent`.
  - Keine schwebenden Steine im finalen Board.
  - Deterministisches Replay derselben Sequenz gegen ein zweites Repository.

  Warum dieser Test relevant ist:
  Der Test zeigt nicht nur einzelne Funktionen, sondern das Zusammenspiel des Event-Projektionsmodells. Besonders wichtig ist das Replay: Dieselbe Sequenz wird erneut auf ein unabhängiges Repository angewendet und anschließend wird der serialisierte Board-Zustand verglichen. Damit wird geprüft, dass der Zustand reproduzierbar aus dem Event-Stream entsteht.

  Zentrale Assertion:

  ```
  serializeBoard(state1) == serializeBoard(state2)
  ```

6.2 Konsolenausgaben

Die Konsolenausgaben dienen als ergänzender visueller Nachweis der Projektion. Nach jedem angewendeten Domain-Event gibt der Client Event-Typ, `gameId`, Status und Board aus. Dadurch ist nachvollziehbar, dass der lokale Zustand nicht durch Konsoleneingaben oder clientseitige Spiellogik entsteht, sondern ausschließlich durch bestätigte Events.

Spielstart:

```
=== EVENT APPLIED: GameStartedEvent ===
GameId: 03704771-49c7-408b-aec8-6e186e17f69d
Status: RUNNING
Board:
. . . . . . . .
. . . . . . . .
. . . . . . . .
. . . . . . . .
. . . . . . . .
. . . . . . . .
```

Bestätigte Züge:

```
=== EVENT APPLIED: MovePlacedEvent ===
GameId: 03704771-49c7-408b-aec8-6e186e17f69d
Status: RUNNING
Board:
. . . . . . . .
. . . . . . . .
. . . . . . . .
. . . . . . . .
. O . . . . . .
X O X O X O X O
```

Reihenlöschung:

```
=== EVENT APPLIED: BottomRowDeletedEvent ===
GameId: 03704771-49c7-408b-aec8-6e186e17f69d
Status: RUNNING
Board:
. . . . . . . .
. . . . . . . .
. . . . . . . .
. . . . . . . .
. . . . . . . .
. O . . . . . .
```

Spielgewinn:

```
=== EVENT APPLIED: GameWonEvent ===
GameId: 03704771-49c7-408b-aec8-6e186e17f69d
Status: WON
WINNER: PLAYER1
Board:
. . . . . . . .
X . . . . . . .
O . . . . . . .
X . . . . . . .
O . . . . . . .
X O . . . . . .
```

Beobachtung:
Die Ausgabe zeigt, dass jeder angewendete Event einen nachvollziehbaren Zustandsübergang erzeugt. Besonders wichtig ist, dass der Gewinn nicht aus dem Board berechnet wird, sondern erst nach dem `GameWonEvent` sichtbar wird. Ebenso wird die Reihenlöschung erst nach dem entsprechenden Event durchgeführt.

7 Erfolgreicher Spielverlauf

7.1 Verwendete gameId

`03704771-49c7-408b-aec8-6e186e17f69d`

7.2 Chronologische Eventliste (realer Verlauf)

1. Spielstart:

```
GameStartedEvent(gameId, rows=6, cols=8)
```

2. Erste Zugsequenz:

```
MovePlacedEvent(gameId, PLAYER1, column=1)
MovePlacedEvent(gameId, PLAYER2, column=2)
MovePlacedEvent(gameId, PLAYER1, column=3)
MovePlacedEvent(gameId, PLAYER2, column=4)
MovePlacedEvent(gameId, PLAYER1, column=5)
MovePlacedEvent(gameId, PLAYER2, column=6)
MovePlacedEvent(gameId, PLAYER1, column=7)
MovePlacedEvent(gameId, PLAYER2, column=8)
```

3. Reihenlöschung:

```
BottomRowDeletedEvent(gameId)
```

4. Vertikale Aufbauphase:

```
MovePlacedEvent(gameId, PLAYER1, column=1)
MovePlacedEvent(gameId, PLAYER2, column=1)
MovePlacedEvent(gameId, PLAYER1, column=1)
MovePlacedEvent(gameId, PLAYER2, column=1)
MovePlacedEvent(gameId, PLAYER1, column=1)
```

5. Spielgewinn:

```
GameWonEvent(gameId, PLAYER1)
```

7.3 Finaler Zustand

```
Status: WON
Winner: PLAYER1
```

Systemische Einordnung:

```
Client-Zustand = fold(geordnete OK-Events)
```

Nicht Bestandteil dieser Projektion sind:

- lokale Gewinnprüfung,

- lokale Zugvalidierung,
- inhaltsbasierte Deduplizierung,
- Reordering von Events.

Diese Einschränkungen stellen eine bewusste architektonische Abgrenzung dar und ergeben sich direkt aus der Aufgabenstellung: Der Game-Master übernimmt Validierung und Entscheidungslogik, Kafka gewährleistet die geordnete Event-Kommunikation pro Partition, und der Client projiziert den bestätigten Event-Strom deterministisch auf sein lokales Modell.

8 Antworten zu Kafka-Grundlagen

8.1 Datenorganisation in Topics

Die Kommunikation erfolgt über zwei logisch getrennte Kafka-Topics: `game-requests` (Client → Game-Master) und `game-events` (Game-Master → Clients).

Ein Topic ist physisch in Partitionen unterteilt, wobei jede Partition ein append-only Log mit strikt monoton steigenden Offsets darstellt. Die Reihenfolge von Nachrichten ist ausschließlich innerhalb einer Partition garantiert.

Die Nachrichten sind JSON-serialisiert. Dabei gilt:

- Requests enthalten genau eine fachliche Aktion (z. B. `newGame`, `move`)
- Events werden als `GameEventEnvelopeDTO` übertragen und enthalten:
  - `gameId` als fachlicher Schlüssel
  - `state` (`OK` oder `ERROR`)
  - `actions` als Liste von Aktionen

Ein einzelnes Kafka-Event kann mehrere fachliche Events enthalten. Diese werden im Client durch den `EventMapper` deterministisch in Domain-Events transformiert und sequenziell verarbeitet.

8.2 Partitionierung

Die Partitionierung erfolgt über den Kafka-Key. Im Client wird konsequent die `gameId` als Key verwendet:

```
producer.send(dto, command.getGameId().toString());
```

Kafka berechnet die Zielpartition deterministisch über:

$$\text{partition} = \text{hash}(\text{key}) \bmod \text{Anzahl Partitionen}$$

Daraus folgt:

- Alle Events eines Spiels werden in derselben Partition gespeichert
- Die Reihenfolge der Events bleibt erhalten (Ordering-Garantie)
- Der Spielzustand kann deterministisch durch Event-Projektion rekonstruiert werden

Ohne Key-basierte Partitionierung würden Events eines Spiels auf mehrere Partitionen verteilt werden, wodurch die kausale Reihenfolge verloren geht.

8.3 Consumer Groups

Jede Client-Instanz verwendet eine eigene Consumer Group:

```
GROUP_ID = "game-client-" + UUID.randomUUID()
```

Konsequenz:

- Jeder Client erhält den vollständigen Event-Stream (Publish-Subscribe)
- Clients sind unabhängig voneinander und rekonstruieren ihren Zustand separat

Für skalierbare Verarbeitung innerhalb einer Consumer Group gilt:

- Maximale Parallelität = Anzahl Partitionen
- Für bis zu 10 parallel arbeitende Consumer in einer Gruppe werden mindestens 10 Partitionen benötigt
- Wenn weniger Partitionen existieren als Consumer, bleiben überschüssige Consumer inaktiv

8.4 Vermeidung mehrfacher Verarbeitung

Kafka garantiert standardmäßig nur *at-least-once* Zustellung. Mehrfache Verarbeitung kann insbesondere bei Consumer-Abstürzen oder Rebalancing auftreten.

Folgende Maßnahmen wurden implementiert:

- Deaktivierung von Auto-Commit:

  ```
  ENABLE_AUTO_COMMIT = false
  ```

- Explizites Commit nach erfolgreicher Verarbeitung:

  ```
  consumer.commitSync();
  ```

- Deduplizierung über eindeutige Record-ID:

  ```
  recordId = topic + partition + offset
  ```

- Speicherung bereits verarbeiteter Offsets:

  ```
  processedRecords.add(recordId)
  ```

Dadurch wird sichergestellt, dass ein Event trotz möglicher Mehrfachzustellung nicht mehrfach auf den lokalen Zustand angewendet wird.

8.5 Standalone Consumer

Ein Standalone Consumer arbeitet ohne Consumer Group und verwendet typischerweise `assign()` statt `subscribe()`.

Unterschiede:

- Keine automatische Partition-Zuweisung (kein Rebalancing)
- Kein Failover bei Ausfall
- Volle Kontrolle über Partitionen und Offsets

Typische Einsatzfälle:

- Debugging einzelner Partitionen (gezieltes Lesen bestimmter Offsets)
- Replay von Events zur Zustandsrekonstruktion
- Monitoring und Analyse von Event-Strömen

Für das Spielsystem sind Consumer Groups notwendig, da mehrere Clients parallel und unabhängig arbeiten sollen.

9 Probleme und Lösungen

- **Behandlung von ERROR Events**
  Problem: Der Game-Master kann Requests ablehnen, wodurch Events mit `state = ERROR` entstehen.
  Lösung: Diese Events werden nicht auf den Zustand angewendet, sondern nur ausgegeben:

  ```
  if ("ERROR".equals(dto.getState())) {
      eventMapper.printError(dto);
      continue;
  }
  ```

- **Synchronisation mehrerer Clients**
  Problem: Mehrere Clients müssen konsistent denselben Spielzustand rekonstruieren.
  Lösung:
  - Verwendung der `gameId` als Partition-Key
  - Filterung nach `gameId` im Consumer
  - Zustandsaufbau ausschließlich durch bestätigte Events

- **Asynchrone Eventverarbeitung**
  Problem: Eingabe (User) und Event-Verarbeitung laufen parallel.
  Lösung:
  - Trennung in Consumer-Thread und Input-Thread
  - Synchronisation über `GameContext` und `CountDownLatch`
  - Verarbeitung erst nach gesetzter `gameId`

- **Event-Reihenfolge und Konsistenz**
  Problem: Falsche Reihenfolge führt zu inkorrektem Zustand.
  Lösung:
  - Deterministische Partitionierung über `gameId`
  - Sequenzielle Verarbeitung innerhalb der Partition
  - Keine lokale Validierungslogik, sondern reine Event-Projektion

- **Mehrfache Event-Zustellung**
  Problem: Events können erneut geliefert werden (z. B. nach Rebalance).
  Lösung:
  - Offset-basierte Deduplizierung
  - Manuelles Commit nach Verarbeitung

- **Initialisierung des Spielzustands**
  Problem: Events können vor Initialisierung eintreffen.
  Lösung:

  - Zustand wird ausschließlich durch `GameStartedEvent` erzeugt
  - Alle anderen Events werden bis dahin ignoriert

- **Ungültige Eingaben**
  Problem: Fehlerhafte User-Eingaben (z. B. ungültige UUID oder Spalte).
  Lösung:
  - Validierung im `ConsoleInputController`
  - Fehlerbehandlung ohne Systemabbruch

10 Fazit

Die Umsetzung bildet die Anforderungen der Aufgabenstellung konsistent im Sinne einer eventgetriebenen Architektur ab. Der lokale Spielzustand entsteht ausschließlich durch die sequenzielle Verarbeitung bestätigter Events des Game-Masters, wodurch eine klare Trennung zwischen Entscheidungslogik und Zustandsprojektion gewährleistet ist.

Die Verwendung der gameId als Kafka-Key stellt sicher, dass alle Events eines Spiels innerhalb einer Partition verarbeitet werden und damit eine kausal korrekte Reihenfolge erhalten bleibt. Dies ermöglicht eine deterministische Rekonstruktion des Spielverlaufs aus dem Event-Stream.

Nebenläufigkeit wird gezielt durch die Trennung von Eingabe- und Eventverarbeitung umgesetzt, ohne zusätzliche Komplexität einzuführen. Gleichzeitig adressiert die Implementierung typische Aspekte verteilter Systeme wie at-least-once-Zustellung, Offset-Management und die Behandlung fehlerhafter Events.

Die Architektur folgt konsequent dem Prinzip, dass bestätigte Events die einzige Grundlage des Systemzustands darstellen, und bleibt damit reproduzierbar, konsistent und unabhängig von lokaler Logik.